[SSC26-VII-08]

Lightweight Open Source On-Spacecraft Machine Learning with mlpack and F Prime

Ryan R. Curtin
NumFOCUS, Inc.
ryan@ratml.org

Scott M. Gilliland
Georgia Institute of Technology
scott.gilliland@gatech.edu

Sterling L. Peet
Georgia Institute of Technology
sterling.peet@gatech.edu

ABSTRACT

Onboard machine learning is an increasingly important topic in spaceflight systems, as machine learning and AI models are increasingly used for data processing, anomaly detection, attitude control systems, and other predictive applications. However, successfully deploying a machine learning model in a spaceflight computing environment is fraught with difficulty, as spaceflight computing environments are low-resource, but typical machine learning software solutions are implemented in languages like Python, which require an interpreter and other heavyweight dependencies. It is therefore virtually mandatory for any on-spacecraft machine learning to be written with lightweight toolkits that minimize dependencies and resource usage.

We demonstrate a solution to this problem: via the use of open source C++-based scientific computing software, including the mlpack machine learning library and Armadillo linear algebra library, we provide the first implementation of a working on-spacecraft machine learning anomaly detection system inside the F Prime flight software framework that successfully detects non-trivial anomalous events by using a kernel density estimate on telemetry data. Our prototype is validated on real hardware, where we physically created several anomalous situations that were all successfully detected. The code is available publicly on Github and can be used as a reference project for deploying any complex machine learning pipeline to a spacecraft via the F Prime framework.

1 INTRODUCTION

Modern spaceflight missions produce massive amounts of data. NASA missions are now generating over 100 TB of data each day,[1] and this presents a significant challenge for data processing. Rather than downlinking the data for ground-based processing, many are turning to direct on-board machine learning to process the data.[2–4] The limited computational resources available on low-power satellites, such as CubeSats[5] and other similar craft such as extraplanetary rovers pose challenges to complex onboard processing.

Due to the focus on flight heritage for the space applications, the chosen computational platforms of satellites can be somewhat esoteric. Although ARM-based platforms are likely the most common, with use in missions like SpaceCube[6] and DODONA,[7] legacy platforms are also in wide usage. For instance, the ArgoMoon mission[8] and Roman Space Telescope[9] use SPARC processors. The common BAE RAD750[10] is a PowerPC processor and is used on the Perserverance and Curiosity rovers.[9] Even platforms like MIPS are in use on missions like New Horizons.[11]

A further constraint on spaceflight computing systems is the flight software framework being used to control them. Two common choices, both supported by NASA, are F Prime[12] and core Flight System (cFS).[13] cFS is written in C, adhering to the C99 standard. Although this allows for precise programmer control in an embedded context, it makes interoperability with non-C99 toolkits difficult. For similar reasons, F Prime is written in C++, targeting the C++11 standard or newer. The interoperability situation is better for C++, but can still present

some challenges. Due to limited computational resources, and to reduce complexity, it is often best to keep all computation directly in the language of the flight software toolkit.

This variety of computing platforms, the general lack of computing resources present on them, as well as the language limitations imposed by cFS and F Prime present a huge barrier to practitioners wishing to perform machine learning directly on a spacecraft. Most widely-used machine learning software packages, such as `scikit-learn`,[14] TensorFlow,[15] or PyTorch[16] provide interfaces in Python. This is problematic for lightweight deployment: Python requires a language interpreter, which is inefficient and can be orders of magnitude larger than storage resources permit.[17] Even projects aimed to productionalize machine learning models present problems: efforts like XLA,[18] TensorFlow Lite, and ExecuTorch for PyTorch are cumbersome to build and link against, as they use complex build systems and have many dependencies. Vendor-specific alternatives like ARM's CMSIS-NN[19] suffer from limited vendor support, limited functionality, and cannot be deployed on other hardware. Worse, vendor-supplied toolkits are often arbitrarily EOL'ed, as in the case of several Qualcomm and Intel toolkits. Another barrier encountered with these tools is complexity: software running on-spacecraft is expected to be highly reliable, stable, and maintainable.

Yet there exists a widely-used native C++ scientific computing ecosystem, made up of tools like Armadillo,[20] Eigen,[21] and xTensor[22] for linear algebra, mlpack,[23] dlib-ml,[24] and ensmallen[25] for machine learning, and numerous other libraries for related tasks. Many of these tools are intentionally lightweight, header-only, and dependency-free; thus they can be easily cross-compiled to any target architecture and deployed without burdensome dependency requirements. These libraries have been deployed widely, including for other on-board spaceflight applications.[26,27]

In this paper, we put both of these pieces together to solve the problem: we demonstrate that mlpack can be easily used inside of F Prime to build a lightweight machine learning anomaly detector that runs directly on-board the spacecraft without the need for ground-based intervention. Following Peet et al.,[28] we deployed our solution to a Pololu Romi robot,[29] which serves as a surrogate of a small satellite or extraplanetary rover. Our anomaly detector framework was able to successfully detect a number of complex anomalous situations that are not trivially detected with simple methods, e.g., thresholding.

Although our purpose here is to detect telemetry anomalies, our solution can be trivially adapted to other common on-board machine learning tasks, such as computer vision and image classification,[30,31] health monitoring,[32] and other tasks. Our code is publicly available for use and adaptation at `https://github.com/SterlingPeet/mlpack-fprime-robot`, licensed under the Apache license, version 2.[33]

2 SOFTWARE OVERVIEW

Our flight control software is implemented in C++ as an F Prime deployment that internally uses mlpack's kernel density estimation technique for its anomaly detection functionality.

2.1 F Prime

As mentioned earlier, F Prime is a modular flight software framework emphasizing the reusability of individual parts of software across different missions. In F Prime, each part of the flight software is decomposed into discrete *Components* with well-defined interfaces; these serve as the fundamental building blocks for the mission software.

A single software executable consists of a number of components connected in a specific way, called a *deployment*. Each F Prime component's interface is defined as a number of input and output ports, which are defined and connected to other components. This interface and connections are specified in the Fpp domain-specific language.[34] For example, the `Svc.SystemResources` component, which monitors the state of the computing environment (e.g. RAM usage, CPU utilization), typically outputs its telemetry via an Fpp connection to the mission telemetry service for relay to ground control systems.

The F Prime framework encapsulates all of the communication between components, and provides numerous default components and deployments. This means that for a user, implementing the mission software stack typically means taking several F Prime-supplied off-the-shelf components and connecting them with a few custom components implemented specifically for the mission.

Once the components are implemented and the connections in the deployment are defined, an F Prime deployment can easily be cross-compiled to any target platform, including traditional platforms like VxWorks on the RAD750, and newer platforms like ARM systems running lightweight Linux distributions (for instance, a Raspberry Pi).

Compilation and configuration is handled via CMake, which is likely the most widely-used C++ build system generator. F Prime automatically generates the CMake configuration based on the deployment topology. A user who has implemented their flight software with F Prime can typically build and run it with a few simple commands via the use of the helper programs `fprime-util` and `fprime-gds`:

```
# automatically configure CMake
$ fprime-util generate

# build project with CMake
$ fprime-util build

# run the software and display
# ground control interface
$ fprime-gds
```

F Prime deployments can work even on very resource-constrained hardware; for instance, it has been successfully used with program memory size as small as 128 kB and RAM as small as 64 kB.[35,36]

Further resources on F Prime can be found in the original F Prime paper[12] and website,[37] and more details related to our deployment topology can be found in the previous work of Peet et al.,[28] whose work we have used as a starting point.

2.2 mlpack

mlpack is a lightweight header-only machine learning library written in C++ with a focus on portability and efficiency.[23] It is built on top of the ensmallen mathematical optimization library,[25] the cereal serialization library,[38] and the well-known Armadillo linear algebra library[20] for its linear algebra primitives. It can also make use of the more recent Bandicoot library[39] to provide support for GPU-accelerated linear algebra operations. All of these libraries take advantage of C++'s template metaprogramming support to provide significant runtime accelerations and size reductions.

Importantly, mlpack and all of its dependencies are header-only, with the sole exception of Armadillo, which must be linked against a BLAS/LAPACK library. This significantly eases the compilation process, as users simply need to ensure that the headers are available. Although users must supply a compiled BLAS/LAPACK library specific to the target architecture, the OpenBLAS library[40] has easy support for cross-compilation to a wide variety of targets (including all spacecraft architectures described in the introduction), and mlpack has a CMake-based auto-downloader that can obtain all of the header-only dependencies of mlpack and automatically compile or cross-compile OpenBLAS without requiring user intervention.

From a high level, using mlpack for machine learning purposes is generally isomorphic to the workflows used in Python-based toolkits such as `scikit-learn`. Linear algebra is performed with Armadillo (or Bandicoot), where the API is expressly meant to mirror MATLAB to make transition to C++ easier for researchers and practitioners. For example, this simple code snippet creates two random uniform matrices of size 100×50 and computes a third as $Z = (X + Y)/2$.

```
// For more details on the Armadillo API, see
// https://arma.sourceforge.net/docs.html.
arma::mat X(100, 50, arma::fill::randu);
arma::mat Y(100, 50, arma::fill::randu);
arma::mat Z = (X + Y) / 2;
```

Typical classification and regression algorithms in mlpack implement a simple and unified API. For example, supposing we wish to build a decision tree on a dataset of observations X and a vector of labels Y, we can use the following code.

```
// This matrix holds the predictors.
arma::mat X;
// This vector holds the labels.
arma::Row<size_t> Y;

// Create the decision tree, optionally
// specifying parameters.
mlpack::DecisionTree d(/* ... */);
d.Train(X, Y);
```

The `d.Classify()` function could then be used to predict the class of a new data point using the trained decision tree. There are also a number of hyperparameters that control the tree-building procedure that can be specified in the constructor, as well as other methods that can be used to inspect or use the tree in other ways.

Other algorithms in mlpack follow a very similar construct-train-use paradigm, popularized originally by `scikit-learn`[41] and roughly reflected in virtually every modern machine learning and data science library. Available algorithms in mlpack range from common decision trees and logistic regression to more specialized tree-based algorithms[42] including tree-based density estimation techniques,[43,44] which is what we have chosen to use here for our on-spacecraft anomaly detector.

It is worth noting that JPL F Prime coding conventions[45] explicitly disallow numerous C++ fea-

tures including dynamic memory allocation after initialization. With mlpack and related tools, it is possible to meet each of these conventions but extra work may be required. For example, F Prime components that use Armadillo matrices could either use `arma::mat::fixed<Rows, Cols>` to allocate a fixed-size matrix at compile time, or could call `mat.set_size(rows, cols)` at F Prime component construction time to perform dynamic allocation only once. Other approaches exist too, e.g., a custom allocated-once memory pool used during computation. mlpack offers a few compile-time controls to assist with this, including macros that allow disabling various parts of STL functionality.

3 HARDWARE OVERVIEW

Building on Peet et al.,[28] we choose the inexpensive Pololu Romi robot[29] (shown in Figure 1) as a robotic explorer platform due to its standard hardware: an ATMega32U4-based mainboard controlled by a Raspberry Pi via an I^2C connection; in our case, we used a Raspberry Pi 3B+ which contains a Cortex-A53 processor at 1.4 GHz and 1GB of RAM. F Prime (and the machine learning anomaly detector) runs on the Raspberry Pi, and components in the topology interact with the ATMega32U4 via I^2C.

From a control perspective, the Romi contains two wheels with encoders for speed control, a speaker, and 3 LEDs (which we do not use for our work here). From a telemetry perspective, the following signals are available on the Romi and can be captured for F Prime telemetry:

- Wheel turn counters (can be used for distance tracking)
- Acceleration sensors (x/y/z axis)
- Gyrometers (x/y/z)
- Ambient temperature
- Battery level

Figure 1: Pololu Romi robot.

This is in addition to the default F Prime telemetry running on the Raspberry Pi, which can track available memory, memory used, and CPU utilization percentages.

Although the Romi seems unlikely to survive for very long if deployed as-is in a spaceflight environment, we find it to be a compelling demonstration platform, as it reflects typical qualities of real spaceflight computing platforms: it has limited computational power and memory, as well as multiple interconnected devices, sensors, and motors that enable (semi-)autonomous operation, and its main processor architecture (ARM) is commonly used in COTS spaceflight applications.

4 OPERATING CONCEPT

The operating concept of our robotic explorer envisions two distinct operating settings:

- **Stationary**. In this mode, the explorer does not use its wheels to move, and it passively collects environmental data. It is expected in this mode that neither the external environment (measured via temperature, acceleration) nor the internal environment (CPU usage) changes significantly.
- **Straight-line**. In this mode, the explorer is always moving forward or backward at any low-to-medium velocity, or is stopped. It is expected that the measured rate of travel matches the controlled wheel speeds, and that the controlled and measured velocities are not a high velocity.

During normal operation, an anomaly detection system that tracks previous telemetry is always running, and should the explorer enter an operating region where the collected telemetry does not match telemetry collected from known-good operation, an error will be issued. In our implementation, we simply used the speaker to indicate anomalies audibly. (In a real spaceflight application, it is likely that either a louder speaker or some other non-acoustic anomaly reporting means would be desired.)

Commands sent to the explorer can set it in any of these two modes, and reset the anomaly detection system such that its recent collected telemetry is interpreted as corresponding to the explorer working as desired.

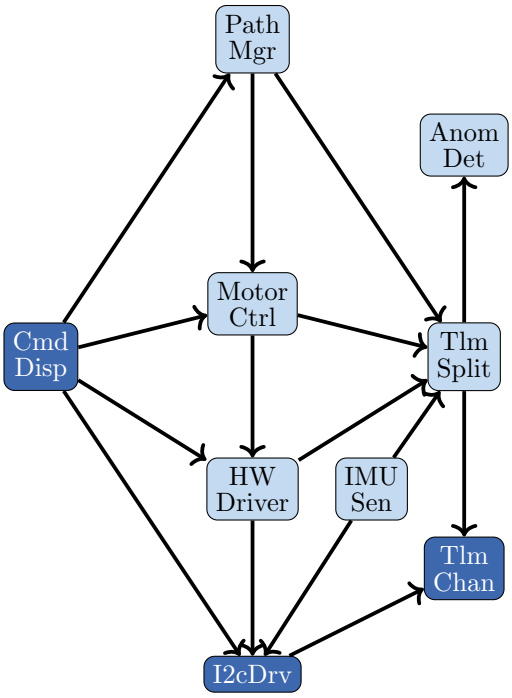

Figure 2: F′ architecture for our robotic explorer.

5 ARCHITECTURE

In order to support the two modes of operation desired for our robotic explorer, we architect F Prime components as shown in Figure 2. Each component already provided by F Prime is printed in dark blue, while components developed specifically for our mission requirements are in light blue. A few less important connections between components are omitted for brevity. A description of each component's responsibilities is below:

- **CmdDisp**: dispatches incoming operator commands to the relevant components; included in F Prime.
- **PathMgr**: overall control; issues commands to put the explorer in one of the two path modes.
- **MotorCtrl**: motor control; sets the motor speeds and computes odometry.
- **AnomDet**: density-estimation anomaly detector implemented with mlpack; receives telemetry from TlmSplit.
- **HWDriver**: Romi hardware driver; issues hardware commands via I2cDrv to I^2C.
- **IMUSen**: Romi IMU sensor driver; issues hardware commands via I2cDrv to I^2C.
- **I2cDrv**: low-level I^2C driver included in F Prime.
- **TlmSplit**: telemetry splitter; routes all telemetry *both* to default TlmChan and also AnomDet.
- **TlmChan**: telemetry collector included in F Prime.

Note that because F Prime restricts outputs from components to be routed to only one destination component, we implemented a telemetry splitter (TlmSplit) with one input port for input telemetry and two output ports. This allows us to duplicate all telemetry to both the default F Prime telemetry system and our custom anomaly detector.

6 ANOMALY DETECTION

Historically, anomaly detection was performed on spacecraft via simple thresholds applied to telemetry values.[46] However, this simple approach suffers from many false positives, needs tedious and error-prone tuning of the thresholds, and cannot easily detect situations where each individual telemetry value is within tolerance, but combinations of values indicate a malfunction. (Consider, as we will later, the trivial case of a wheel's rate of motion not matching the power applied to it.)

Of course, more recent efforts have used far more complex techniques to detect anomalies, including LSTMs (neural networks),[47] convolutional neural networks,[48] and kernel methods,[49] to name a few. Of note is that the majority of these efforts are applying machine learning on the ground, instead of directly on the spacecraft.

With F Prime and mlpack, each of the techniques referenced above (and many more) can be deployed and run directly on the spacecraft, presuming that the spacecraft has the computational power necessary to run the technique. For 'traditional' machine learning techniques like a decision tree this is not generally a problem, but neural-network based approaches such as LSTMs can have much higher computational requirements, depending on the size of the network being used.

Because our efforts here are to demonstrate a working prototype, we opt to keep our machine learning simple and easy to understand. We will use a kernel density estimation (KDE) approach to model the probability density of observed telemetry states. Then, the anomaly detection system will

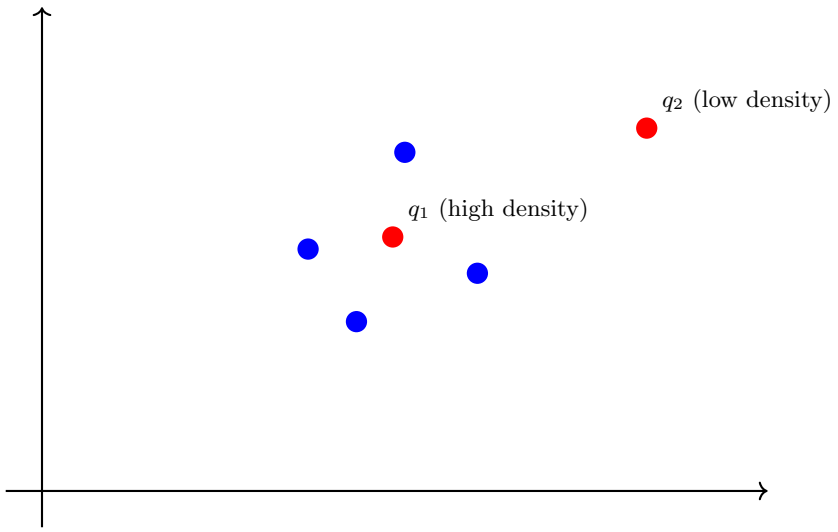

Figure 3: Kernel density estimation. The observed telemetry points (blue) are used to model the distribution (also blue) via convolution with a Gaussian kernel. Then, new telemetry points are compared with the distribution (red). Point q_2 is outside the distribution and may be considered an anomaly.

compute the probability of the current telemetry state, and if that probability is exceedingly low, then the system concludes that we are in an anomalous state and emits an audible error tone via the buzzer.

This approach is best described visually, as in Figure 3. Due to the limitations of the medium, the figure can only show the density estimate in two dimensions; however, as we are receiving numerous channels of telemetry from various components, our density estimator as implemented uses 31 dimensions of telemetry. As this is a demonstration, we manually tune the density limit used to classify as an anomaly. In a real-world application that used this approach, the density should be tuned by collecting large amounts of non-anomalous data and building an accurate model of the distribution.

Implementationally speaking, mlpack's `KDE` class is used to model the distribution. Internally, mlpack uses tree-based algorithms[43] to provide an efficient estimate of the density for a given telemetry point. To keep the tree small on a low-resource spacecraft, we use a configurable moving window of historical telemetry to model the probability density.

We tuned our density estimator to use a Gaussian kernel with bandwidth of 0.38 and used a simple threshold of -20 for the log-density estimate to identify an anomaly. A real-world application of this technique might cross-validate the bandwidth selection, and use labeled anomalies to tune the threshold; but for our purposes (a software deployment demonstration) we do not find that necessary.

7 EXPERIMENTS

In order to validate our approach, we cross-compiled and deployed our F Prime flight software stack on the rover (using `fprime-util generate aarch64-linux` and `fprime-util build aarch64-linux`) and performed a series of real-world experiments, verifying that the anomaly detection system worked successfully in each anomalous situation we created.

7.1 Stationary Operation

As a first test of the system, we trained the density estimate on 10k samples of data where the robot was entirely stationary and receiving no commands. During this time, because the robot was idle, all motor and odometry sensors read zero and CPU usage was minimal.

Once the density estimate was trained, we observed the system functioning without spurious anomaly detections for a few hours. (Our implementation performs anomaly checks at 1 Hz.) Then, we twice launched computationally intensive tasks on the Raspberry Pi. Each time we did this, the anomaly detection system easily detected the situation and emitted audible alerts, as expected.

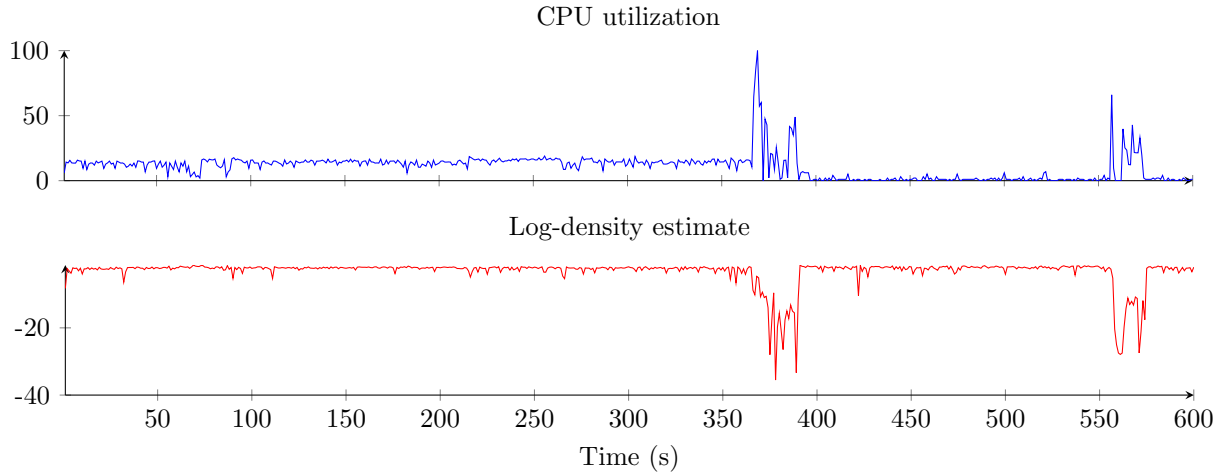

Figure 4: CPU telemetry over time for stationary operation and log-density estimates for anomaly detector. Both manual anomalies were easily detected, when the log-density estimate dropped noticeably for more than five seconds.

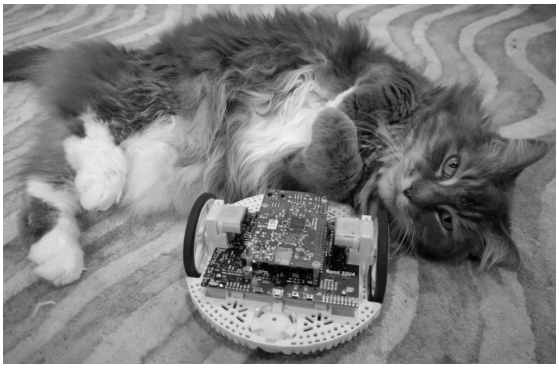

Figure 5: An anomalous situation the rover may encounter in straight-line operation.

Telemetry for CPU1 utilization and the log-density estimate are shown in Figure 4. The two anomalous situations are plainly obvious from the telemetry log; the log-density estimates drop significantly. An interesting observation is the robustness of the density estimator: because we trained it on a few hours of real-world telemetry data, the slightly higher CPU usage before the first anomaly is not itself detected as an anomaly, since this level of CPU usage was regularly seen during the training period, due to e.g. background system services running on the Raspberry Pi.

7.2 Straight-line Operation

The next mode of operation we tested was straight-line operation. In this mode of operation, the rover is expected to be moving forward or backward at low-to-medium speeds. Because the density estimate is trained on 'normal' telemetry, where 'normal' is with respect to the mode of operation, we ran the rover in a clean environment with no obstructions or other problems for approximately 30 minutes. During this time, the rover's speed was set to alternating positive and negative speed values, eventually cycling through all accepted speed values. This process enables the density estimate to properly model all regions of the operating space.

Once the training process was complete, we operated the rover in straight-line mode, setting a variety of different target wheel speeds over time. We also applied six different anomaly conditions:

1. Setting the speed outside the operating region's bounds
2. Obstructing both wheels completely (no movement)
3. Obstructing only one wheel completely
4. Picking up the rover and turning it upside down
5. Picking up the rover and setting it on its side
6. Holding the rover in the air

Figure 6 shows the telemetry and density estimate over this period for a few of the numerous sensors that are connected to the density estimator. In each anomalous situation, the log-density estimate drops precipitously, as expected. Note that the last anomaly (rover in the air) isn't clearly captured by the displayed sensors in the figure; other telemetry values going outside the norm caused the drop in the density.

Overall, these successful detections validate our approach. Although kernel density estimation is

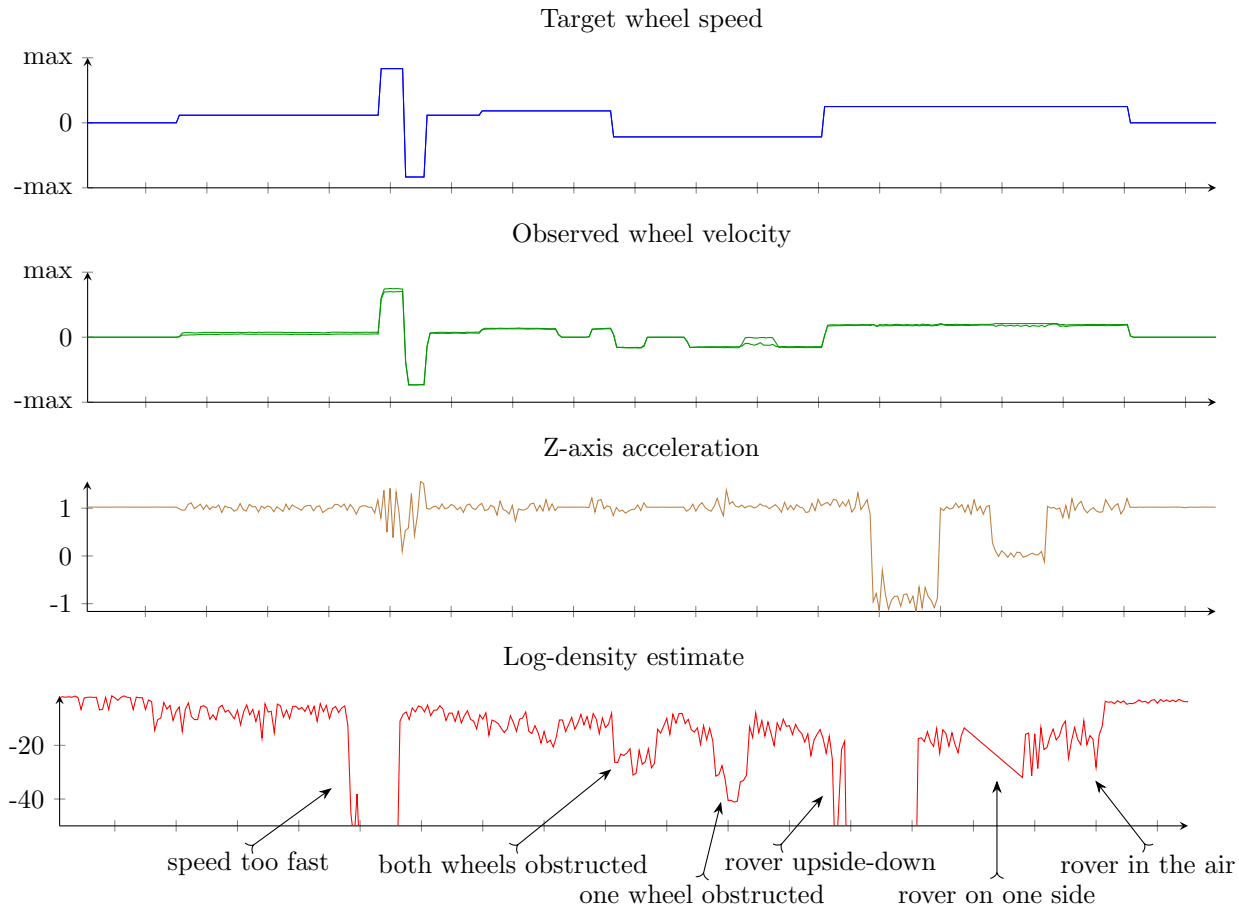

Figure 6: Partial telemetry and log-density estimates for straight-line mode anomaly detection. We applied six different anomalies, each of which caused the log-density estimates to plummet.

not the most powerful known anomaly detector, our aim was simply to demonstrate a working machine learning-based anomaly detection system that was capable of running directly on the spacecraft. Using more complex techniques is left as future work; for now, our publicly available code can be easily used as a starting point.

8 CONCLUSION

We have demonstrated an approach for on-spacecraft machine learning in a resource-efficient and simple way by pairing the F Prime flight software framework with the lightweight mlpack C++ machine learning library. The architecture of F Prime allows an easy way to isolate the machine learning functionality specifically in its own components, meaning that it is easy to drop in place into existing F Prime projects.

In our experiments, we used a simple density estimator to detect anomalies and validated that the mlpack-based model was able to successfully run and detect anomalies on real-world low-resource hardware that might be used in spaceflight missions.

Because mlpack and F Prime are both easy to cross-compile to any target architecture, it would be trivial to recompile our project for, e.g., SPARC64, MIPS, or PowerPC.

Future work to deploy an approach like this to a real-world mission would include adapting parts of mlpack and Armadillo code to follow the JPL style guidelines,[45] and implementing a more robust machine learning-based anomaly detection system.

Our code is publicly available at `https://github.com/SterlingPeet/mlpack-fprime-robot` and is licensed under the Apache license, version 2.0. Telemetry logs of our experiments (and the source code for this paper) are all included in the repository.

References

[1] Jane J. Lee and Ian J. O'Neill. NASA Turns to the Cloud for Help With Next-Generation Earth Missions, 2021.

[2] David Henriquez. High Performance Spaceflight Computing (HPSC). Technical report, Pasadena, CA: Jet Propulsion Laboratory, National Aeronautics and Space Administration, 2016.

[3] Tamer Mekky Ahmed Habib. Artificial intelligence for spacecraft guidance, navigation, and control: a state-of-the-art. *Aerospace Systems*, 5(4):503–521, 2022.

[4] Tyler Cody and Paula J. Dempsey. Application of Machine Learning to Rotorcraft Health Monitoring. Technical report, Glenn Research Center, National Aeronautics and Space Administration, 2017.

[5] Armen Poghosyan and Alessandro Golkar. Cubesat evolution: Analyzing CubeSat capabilities for conducting science missions. *Progress in Aerospace Sciences*, 88:59–83, 2017.

[6] Cody Brewer, Nicholas Franconi, Robin Ripley, Alessandro Geist, Travis Wise, Sebastian Sabogal, Gary Crum, Sabrena Heyward, and Christopher Wilson. NASA SpaceCube intelligent multi-purpose system for enabling remote sensing, communication, and navigation in mission architectures. In *Small Satellite Conference 2020*, number SSC20-VI-07, 2020.

[7] D-Orbit deploys USC's La Jument system cubesat to prove-out space artificial intelligence (AI) technologies, 2024.

[8] Marco Lombardo, Marco Zannoni, Igor Gai, Luis Gomez Casajus, Edoardo Gramigna, Riccardo Lasagni Manghi, Paolo Tortora, Valerio Di Tana, Biagio Cotugno, Simone Simonetti, et al. Design and analysis of the cis-lunar navigation for the ArgoMoon cubesat mission. *Aerospace*, 9(11):659, 2022.

[9] Justin Goodwill, Gary Crum, James MacKinnon, Cody Brewer, Michael Monaghan, Travis Wise, and Christopher Wilson. NASA SpaceCube edge TPU smallsat card for autonomous operations and onboard science-data analysis. In *Proceedings of the Small Satellite Conference*, number SSC21-VII-08. AIAA, 2021.

[10] Richard W Berger, Devin Bayles, Ronald Brown, Scott Doyle, Abbas Kazemzadeh, Ken Knowles, David Moser, John Rodgers, Brian Saari, Dan Stanley, et al. The RAD750: A Radiation Hardened PowerPC Processor for High Performance Spaceborne Applications. In *2001 IEEE Aerospace Conference Proceedings*, volume 5, pages 2263–2272. IEEE, 2001.

[11] David Y. Kusnierkiewicz, Chris B. Hersman, Yanping Guo, Sanae Kubota, and Joyce McDevitt. A description of the Pluto-bound New Horizons spacecraft. *Acta Astronautica*, 57(2-8):135–144, 2005.

[12] Robert Bocchino, Timothy Canham, Garth Watney, Leonard Reder, and Jeffrey Levison. F prime: an open-source framework for small-scale flight software systems. In *32nd Annual AIAA/USU Conference on Small Satellites*, 2018.

[13] David McComas. Nasa/gsfc's flight software core flight system. In *Flight Software WorkshopFlight Workshop*, number GSFC. CPR. 7525.2013, 2012.

[14] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine Learning in Python. *The Journal of Machine Learning Research*, 12:2825–2830, 2011.

[15] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D.G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, 2016.

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, 2019.

[17] Ryan R. Curtin. Lightweight, low-overhead, high-performance: machine learning directly in C++. In *PyData NYC 2023*, 2023.

[18] Amit Sabne. XLA: Compiling machine learning for peak performance. *Google Res*, 2020.

[19] ARM Limited. CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs. https://github.com/ARM-software/CMSIS-NN, 2024.

[20] Conrad Sanderson and Ryan R. Curtin. Armadillo: a template-based C++ library for linear algebra. *Journal of Open Source Software*, 1(2):26, 2016.

[21] Gaël Guennebaud, Benoit Jacob, et al. Eigen. https://eigen.tuxfamily.org, Accessed June 2024.

[22] Johan Mabille, Sylvain Corlay, Wolf Vollprecht, et al. xTensor: multi-dimensional arrays with broadcasting and lazy computing. https://xtensor.readthedocs.io/, Accessed June 2024.

[23] Ryan R. Curtin, Marcus Edel, Omar Shrit and Shubham Agrawal, Suryoday Basak, James J. Balamuta and Ryan Birmingham, Kartik Dutt, Dirk Eddelbuettel and Rishabh Garg, Shikhar Jaiswal, Aakash Kaushik and Sangyeon Kim, Anjishnu Mukherjee, Nanubala Gnana Sai and Nippun Sharma, Yashwant Singh Parihar, and Roshan Swain and Conrad Sanderson. mlpack 4: a fast, header-only C++ machine learning library. *Journal of Open Source Software*, 8(82):5026, 2023.

[24] Davis E King. Dlib-ml: A Machine Learning Toolkit. *Journal of Machine Learning Research*, 10:1755–1758, 2009.

[25] Ryan R. Curtin, Marcus Edel, Rahul Ganesh Prabhu, Suryoday Basak, Zhihao Lou, and Conrad Sanderson. The ensmallen library for flexible numerical optimization. *Journal of Machine Learning Research*, 22(166):1–6, 2021.

[26] Timothy M. Hackett, Sven G. Bilén, Paulo Victor R. Ferreira, Alexander M. Wyglinski, and Richard C. Reinhart. Implementation of a space communications cognitive engine. In *2017 Cognitive Communications for Aerospace Applications Workshop (CCAA)*, pages 1–7. IEEE, 2017.

[27] Timothy M. Hackett, Sven G. Bilén, Paulo Victor R. Ferreira, Alexander M. Wyglinski, Richard C. Reinhart, and Dale J. Mortensen. Implementation and on-orbit testing results of a space communications cognitive engine. *IEEE Transactions on Cognitive Communications and Networking*, 4(4):825–842, 2018.

[28] Sterling L Peet, Scott M Gilliland, and Jeffrey S Young. The framework makes the mission-an analytical comparison of two popular nasa open source flight software framework offerings. In *Proceedings of the Small Satellite Conference 2024*, 2024.

[29] Claire. Pololu - Building a Raspberry Pi robot with the Romi chassis. https://www.pololu.com/blog/663/building-a-raspberry-pi-robot-with-the-romi-chassis, 2017. [Accessed 06-12-2026].

[30] Gianluca Giuffrida, Luca Fanucci, Gabriele Meoni, Matej Batič, Léonie Buckley, Aubrey Dunne, Chris Van Dijk, Marco Esposito, John Hefele, Nathan Vercruyssen, et al. The Φ-Sat-1 mission: The first on-board deep neural network demonstrator for satellite earth observation. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–14, 2021.

[31] Vasileios Leon, Panagiotis Minaidis, George Lentaris, and Dimitrios Soudris. Accelerating ai and computer vision for satellite pose estimation on the intel myriad x embedded soc. *Microprocessors and Microsystems*, 103:104947, 2023.

[32] Al Volponi and Donald L. Simon. Enhanced self tuning on-board real-time model (estorm) for aircraft engine performance health tracking. Technical Report NASA/CR-2008-215272, National Aeronautics and Space Administration, 2008.

[33] Apache Software Foundation. Apache license, version 2.0. https://www.apache.org/licenses/LICENSE-2.0, 2004. [Accessed 06-12-2026].

[34] Robert L Bocchino, Jeffrey W Levison, and Michael D Starch. Fpp: A modeling language for f prime. In *2022 IEEE Aerospace Conference (AERO)*, pages 1–15. IEEE, 2022.

[35] Maximilian Kolhof, William Rawson, Radina Yanakieva, Andrew Loomis, E. Glenn Lightsey, and Sterling Peet. Lessons learned from the gt-1 1u cubesat mission. In *35th Annual Small Satellite Conference*, 8 2021.

[36] Nathan Cheek, Nathan Daniel, E Glenn Lightsey, Sterling Peet, Celeste Smith, and Daniel Cavender. Development of a cots-based propulsion system controller for nasa's lunar flashlight cubesat mission. In *35th Annual Small Satellite Conference*, 8 2021.

[37] F Prime Developers. F prime. https://fprime.jpl.nasa.gov/, 2026. [Accessed 06-12-2026].

[38] W. Shane Grant and Randolph Voorhies. cereal—a C++11 library for serialization. http://uscilab.github.io/cereal, 2017.

[39] Ryan R Curtin, Marcus Edel, and Conrad Sanderson. Fast gpu linear algebra via compile time expression fusion. *arXiv preprint arXiv:2604.22242*, 2026.

[40] Z. Xianyi, W. Qian, and Z. Chothia. OpenBLAS. http://xianyi.github.io/OpenBLAS, January 2021.

[41] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, et al. Api design for machine learning software: experiences from the scikit-learn project. In *European Conference on Machine Learning and Principles and Practices of Knowledge Discovery in Databases*, 2013.

[42] Ryan Curtin, William March, Parikshit Ram, David Anderson, Alexander Gray, and Charles Isbell. Tree-independent dual-tree algorithms. In *International Conference on Machine Learning*, pages 1435–1443. PMLR, 2013.

[43] Alexander G Gray and Andrew W Moore. Nonparametric density estimation: Toward computational tractability. In *Proceedings of the 2003 SIAM International Conference on Data Mining*, pages 203–211. SIAM, 2003.

[44] Parikshit Ram and Alexander G Gray. Density estimation trees. In *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 627–635, 2011.

[45] Code and. https://nasa.github.io/fprime/UsersGuide/dev/code-style.html. [Accessed 06-12-2026].

[46] Sylvain Fuertes, Gilles Picart, Jean-Yves Tourneret, Lotfi Chaari, André Ferrari, and Cédric Richard. Improving spacecraft health monitoring with automatic anomaly detection techniques. In *14th international conference on space operations*, page 2430, 2016.

[47] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 387–395, 2018.

[48] Liang Liu, Ling Tian, Zhao Kang, and Tianqi Wan. Spacecraft anomaly detection with attention temporal convolution networks. *Neural Computing and Applications*, 35(13):9753–9761, 2023.

[49] Ryohei Fujimaki, Takehisa Yairi, and Kazuo Machida. An approach to spacecraft anomaly detection problem using kernel feature space. In *Proceedings of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining*, pages 401–410, 2005.